Month 3: Purple Team Batch 01

Web App, API, & Cloud Security

Week 11: Business Logic Flaws & Advanced Web Attacks

Instructor: **Muhammad Saad**

Server-Side Request Forgery (SSRF)

Server-Side Request Forgery (SSRF) is a type of security vulnerability where a hacker tricks a server into making requests on their behalf.

Imagine a website server is like a receptionist in an office. Normally, it only talks to trusted people or systems. But if someone sends a clever request, they can **trick the receptionist into calling or accessing places it shouldn't**—like private rooms or internal systems.

It's like convincing a bank employee to check a locked vault for you—even though you're not allowed inside—just because you asked in a clever way.

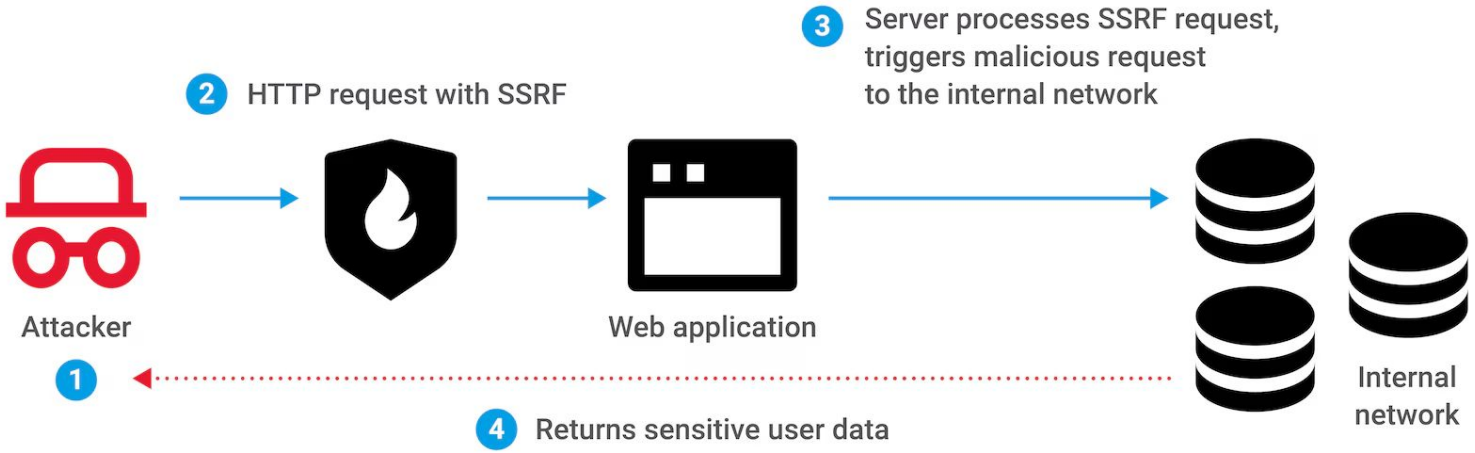
In simple terms:

- The attacker sends a request to a website.
- The website (server) then makes another request based on that input.
- The attacker manipulates this so the server connects to **internal or restricted resources**.

Why it's dangerous:

- The attacker can access **internal systems** that are not exposed to the public.
- They might retrieve **sensitive data** (like passwords or configuration files).
- In some cases, they can even take control of parts of the system.

What is server-side request forgery?



Akamai

XXE (XML External Entity)

XXE (XML External Entity) is a security vulnerability that happens when a system processes XML data in an unsafe way.

Think of XML like a structured message you send to a server. Sometimes, that message can include special instructions called “entities.”

If the server is not properly protected, an attacker can:

- Sneak in a **malicious instruction**
- Trick the server into **reading local files** or **connecting to other systems**

It’s like giving someone a form to fill out, but they secretly add a note saying:

> “Also go open the manager’s safe and tell me what’s inside.”

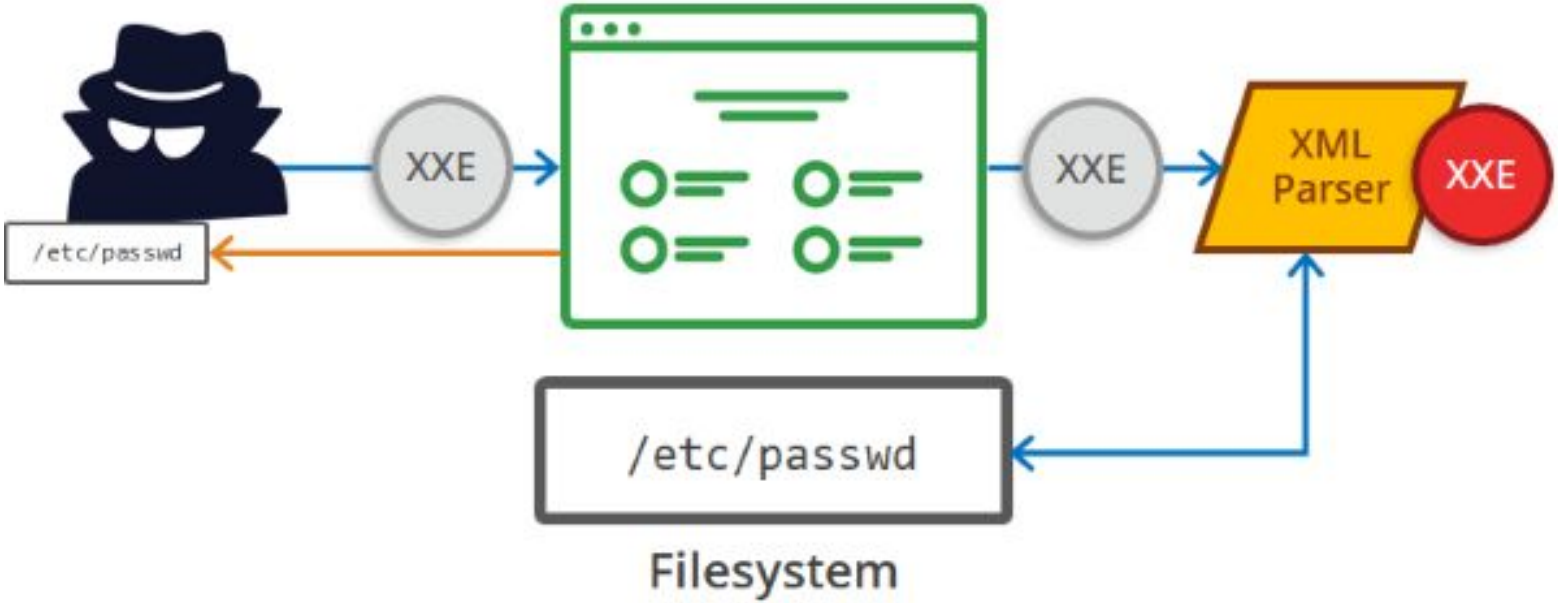
If the employee follows that hidden instruction, sensitive information gets leaked.

In very simple words:

- You send XML data to a server
- The attacker hides a “secret request” inside that XML
- The server unknowingly executes it

Why it’s dangerous:

- It can expose **sensitive files** (like passwords or system data)
- It can be used to attack internal systems (similar to SSRF)
- It may even crash the server


XXE
XXE
XML
Parser
XXE
/etc/passwd
/etc/passwd
Filesystem

Business Logic Flaws

Business Logic Flaws are mistakes in how a system is designed to work — not in the code syntax, but in the *rules and flow* of the application.

A website or app has rules like:

- “You must pay before getting the product”
- “Quantity must be positive”
- “You can only use a discount once”

A **business logic flaw** happens when these rules are **not properly enforced**, so someone can abuse them.

The system works as built… but the *rules are weak or incomplete*, so users can cheat it.

Examples:

- Adding a **negative quantity** in a cart → total price becomes lower or even negative
- Skipping steps → getting a product **without paying**
- Using the same coupon **multiple times**
- Changing the price in a request before sending it

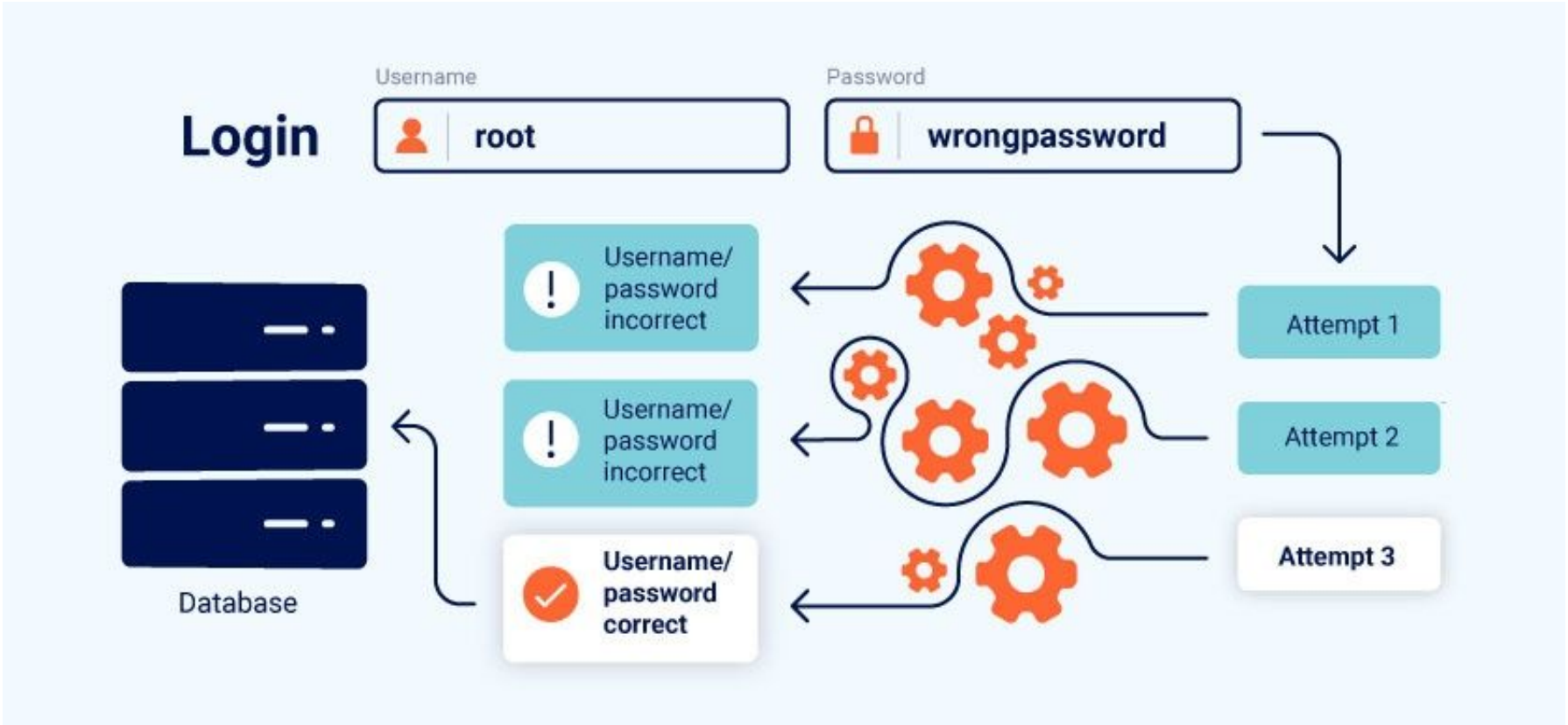
Business Logic Flaws

Why it’s dangerous:

- Users can get products for **free or cheaper**
- Companies lose **money**
- It can break trust in the system

Real-life analogy:

- It’s like a store that says:
- “Take items, then pay at the counter”
- But there’s no one checking if you actually paid — so someone just walks out with the items.


Login
Username
root
Password
wrongpassword
Attempt 1
Attempt 2
Attempt 3
Username/ password incorrect
Username/ password incorrect
Username/ password correct
Database

Insecure Deserialization

Insecure Deserialization is a security problem that happens when a system blindly trusts and processes data it receives, without checking if it's safe.

First, what is "serialization"?

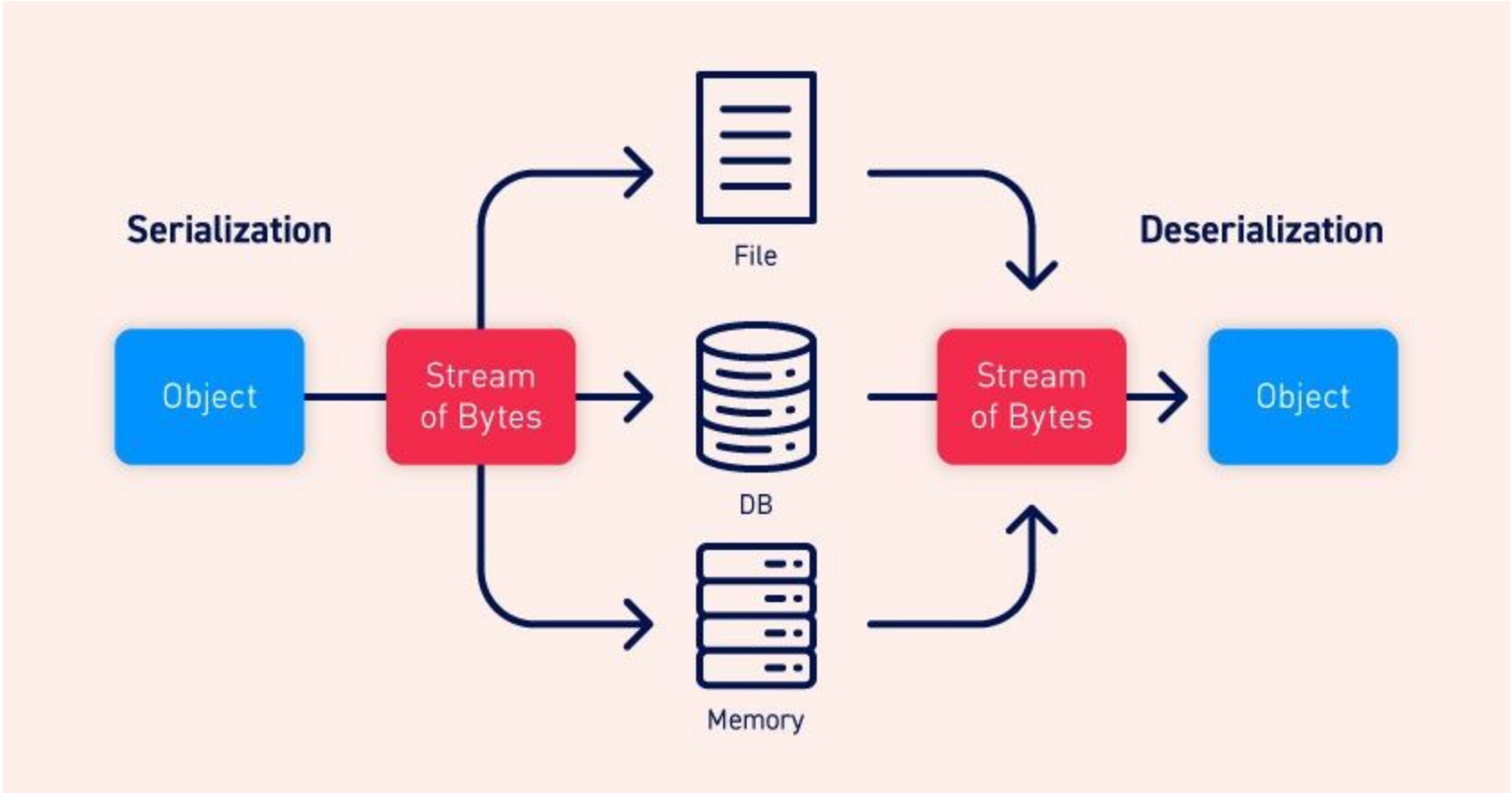
Serialization means converting an object (data) into a format (like text) so it can be:

- Stored
- Sent over the internet

"Deserialization" is turning that data back into an object.

Simple explanation:

- A server receives data (that was previously serialized)
- It converts it back into an object (**deserialization**)
- If an attacker changes that data, the server may **execute harmful instructions**


Serialization
Object
Stream of Bytes
File
DB
Memory
Stream of Bytes
Deserialization
Object

Best of Luck for the Week 11 Tasks

Instructor: **Muhammad Saad**